基于强化学习的游戏决策实验报告

0、测试成功率：

```
  File "E:\Anacorra\envs\rl_env\lib\site-packages\gym\wrappers\order_enforcing.py", line 51, in render
    return self.env.render(*args, **kwargs)
  File "E:\Anacorra\envs\rl_env\lib\site-packages\gym\wrappers\env_checker.py", line 55, in render
    return self.env.render(*args, **kwargs)
  File "E:\Anacorra\envs\rl_env\lib\site-packages\gym\envs\classic_control\acrobot.py", line 361, in render
    self.clock.tick(self.metadata["render_fps"])
KeyboardInterrupt
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\train.py"
Episodes:   0%|                                                                                  | 0/1000 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\DQN.py:91: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider conv
erting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner\_
work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(self.device)
Episodes: 100%|                                                                                  | 1000/1000 [14:14<00:00,  1.17it/s]
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\play.py"
Episodes:   0%|                                                                                  | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:51: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider con
verting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner\
_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                  | 10/10 [05:46<00:00, 34.69s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```

```
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\train.py"
Episodes: 100%|                                                                                  | 1000/1000 [12:14<00:00,  1.36it/s, total_reward=500]
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\play.py"
Episodes:   0%|                                                                                  | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:52: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. P
lease consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Trigge
red internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                  | 10/10 [03:26<00:00, 20.62s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```

```
Episodes:   0%|                                                                                  | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:52: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. P
lease consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Trigge
red internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                  | 10/10 [01:40<00:00, 10.09s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```

1、代码主要思路：

分为DQN.py　train.py　play.py三部分，

一、DQN.py 文件：为算法主体，其中包含：经验回放池、Q网络、DQN

1. 经验回放池（ReplayBuffer）

功能：存储智能体与环境交互的经验数据（状态、动作、奖励、下一状态、终止标志），用于后续随机采样训练，降低数据相关性。

2. Q 网络（Qnet）

- **结构**：单层隐藏层的全连接神经网络，输入为状态维度，输出为动作维度

  输入层→隐藏层（ReLU 激活）→输出层，通过`nn.Linear`定义各层。
- **前向传播**：将状态张量通过隐藏层激活后，输出各动作的 Q 值，用于动作选择和价值计算。

3. DQN算法主体（DQN 类）

- 初始化
  - 创建主网络`qnet`和目标网络`targetnet`，参数相同但更新频率不同。
  - 使用 Adam 优化器，设置学习率、折扣因子`gamma`、探索率`epsilon`等超参数。

- 动作选择（take_action）：
  - **ε- 贪婪策略**：以`epsilon`概率随机选择动作（探索），否则选择 Q 值最大的动作（利用）。
  - 输入状态需转换为张量，通过主网络计算 Q 值并取最大值索引。
- 网络更新（update）：
  - 步骤：
    1. 从经验回放池采样批量数据，转换为张量。
    2. 计算当前 Q 值：通过主网络`qnet`和动作索引`gather`获取对应 Q 值。
    3. 计算目标 Q 值：通过目标网络`targetnet`计算下一状态的最大 Q 值，结合奖励和折扣因子生成 TD 目标。
    4. 均方误差（MSE）计算损失，反向传播更新主网络参数。
    5. 每隔更新频率步，将主网络参数同步到目标网络。

二、train.py 文件：对于模型超参数调整、各种初始化与选项、环境交互奖励积累、模型保存。

1. 超参数设置

- 关键参数：
  - `lr`：学习率，控制网络参数更新幅度。
  - `gamma`：折扣因子，平衡当前与未来奖励（通常取 0.9~0.99）。
  - `epsilon`：探索率，控制随机动作比例（训练初期较高，后期衰减）。
  - `target_update`：目标网络更新频率，避免训练震荡。
  - `buffer_size`：经验回放池容量，存储历史经验。

2. 环境与网络初始化

- 选择 Gym 环境，获取状态和动作空间维度。
- 创建`DQN`实例和经验回放池，指定计算设备（CPU/GPU）。

3. 训练主循环

- 流程
  1. 重置环境，获取初始状态。
  2. 循环交互：根据当前状态选择动作→执行动作获取反馈→存储经验到回放池。
  3. 当回放池数据量≥`minimal_size`时，采样批量数据更新网络。
  4. 记录每回合累计奖励，用于绘制曲线和评估收敛性。

4. 结果保存

- 保存模型参数到文件，绘制奖励曲线并保存，便于后续分析和推理。

三、play.py 文件：配置游戏环境与train.py相同，并调用模型，同时利用Q网络与决策实现与环境交互

1. 参数与网络加载

- 加载训练时的超参数（如`hidden_dim`、环境名称）和模型参数，确保与训练配置一致。
- 初始化 Q 网络并设置为评估模式（关闭 dropout 等训练阶段操作）。

2. 策略函数（policy）

- **纯利用策略**：固定`epsilon=0`（或极小值），仅根据 Q 网络输出选择最大 Q 值动作，无需探索。
- 输入状态转换为张量，通过网络前向传播获取动作索引。

3. 环境交互与可视化

- 重置环境后，循环执行策略动作，渲染环境画面。
- 统计任务成功率，评估模型性能。

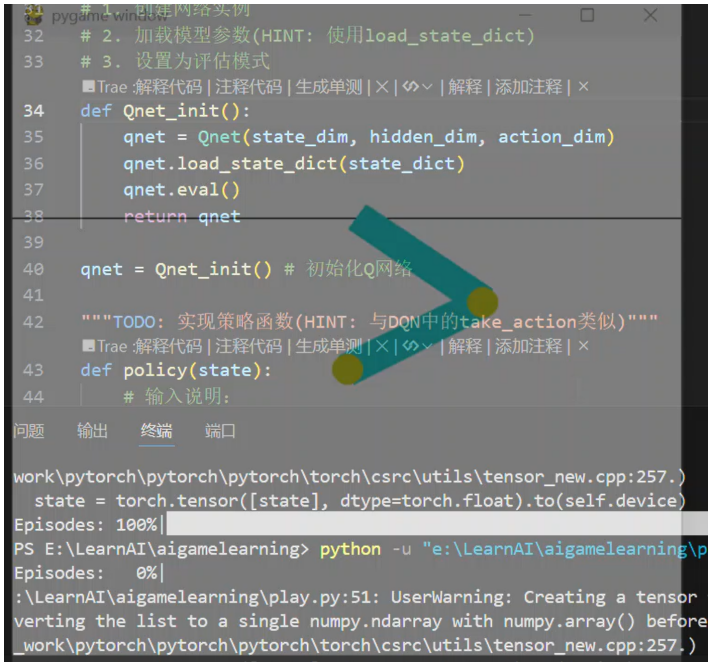
play.py调用train.py训练好模型，以及DQN.py中的Q网络；

train.py调用DQN中Q网络，与经验回放池；

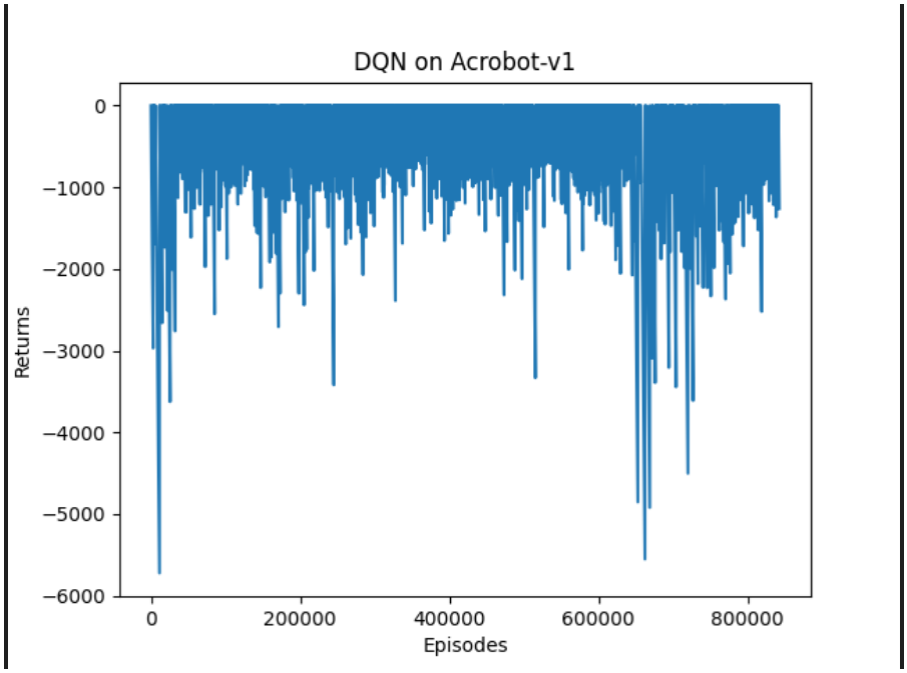
2、实验结果展示与分析：

1.Acrobot

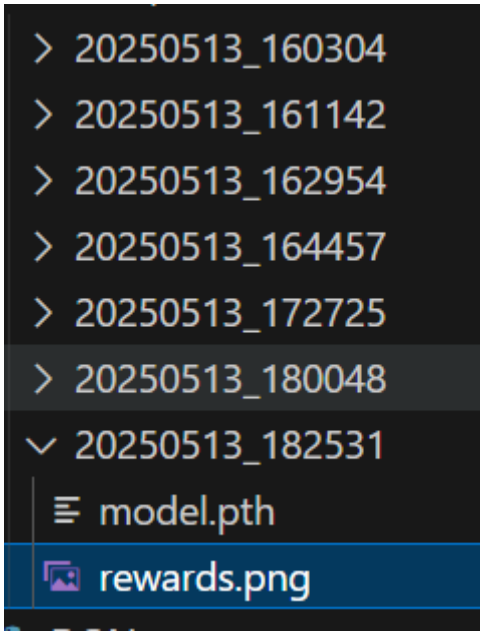
#TODO：视频（Markdown貌似是不支持传入视频的）



（视频中关闭的那一瞬间的帧截取）



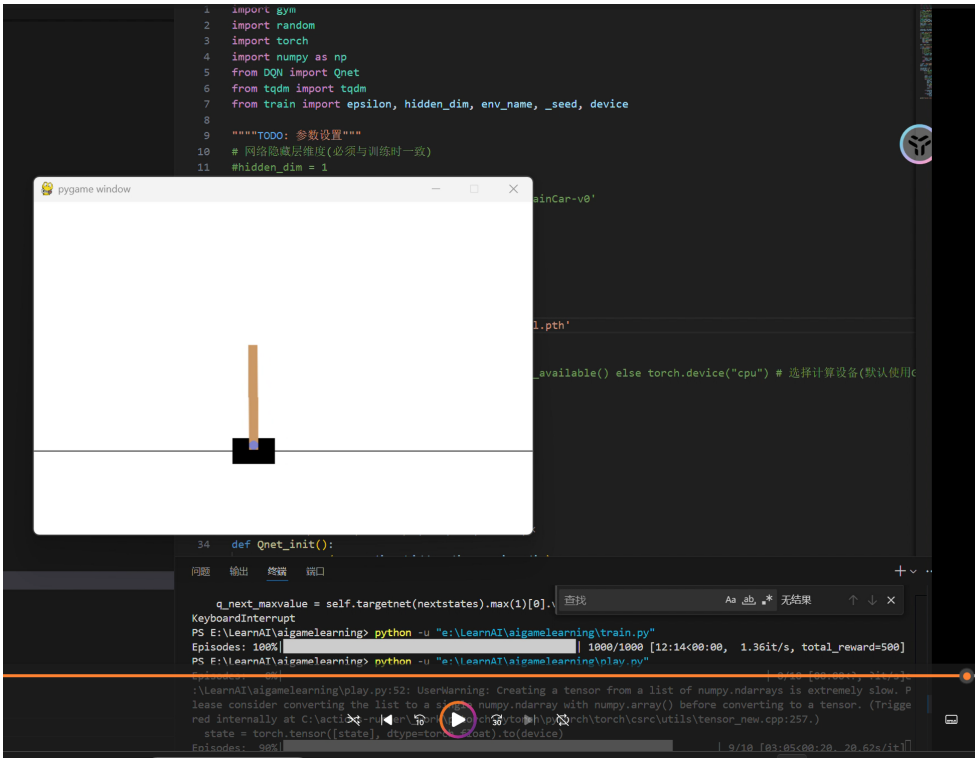
历代模型：



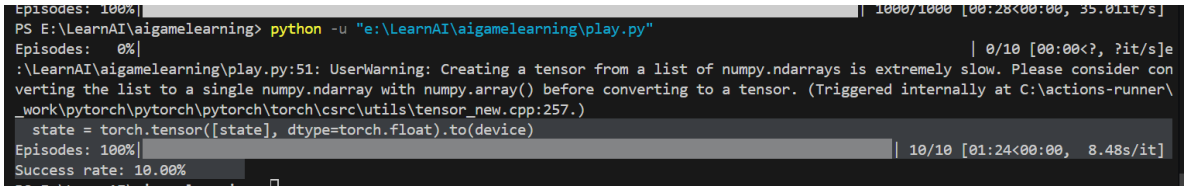

```
"""TODO: 超参数设置区域"""
# -------------------------------
# 学习率
lr = 0.0005
# 总训练回合数
num_episodes = 1000
# 网络隐藏层维度(默认只有一层隐藏层,大家可以根据需求修改)
hidden_dim = 128
# 折扣因子
gamma = 0.95
# 探索率
epsilon = 0.05
# 目标网络更新频率
target_update = 100
# 经验回放池容量
buffer_size = 10000
# 开始训练的最小数据量
minimal_size = 1000
# 批大小
batch_size = 128
# 随机种子(建议固定以复现结果)
_seed = 1553
random.seed(_seed)
np.random.seed(_seed)
torch.manual_seed(_seed)
# -------------------------------
```

2.Cart Pole

~~#TODO: 视频~~ (视频中截图)



200018突然质变，折扣0.95--0.99



target_update = 50 改变，75与25都很明显不行；

EX：

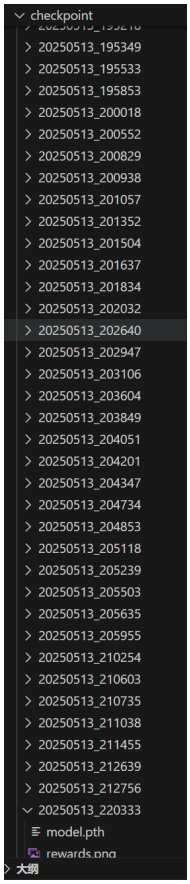
```
"""优化后的超参数设置-用于Cart Pole游戏"""
#
#学习率
-降低学习率可提高训练稳定性
lr=0.0005
#总训练回合数-增加训练回合数使智能体有更多学习机会
num_episodes 2000
#网络隐藏层维度~增大隐藏层维度增强模型表达能力
hidden_dim 128
#折扣因子-短期任务可使用较高的折扣因子
gamma 0.99
#探索率~采用衰减策略平衡探索与利用
epsilon_start 1.0
epsilon_end 0.01
epsilon_decay 0.995
#目标网络更新频率~降低更新频率可提高训练稳定性
target_update 10
#经验回放池容量·增大容量提高数据利用率
buffer_size 50000
#开始训练的最小数据量~积累足够数据后再开始训练
minimal_size 5000
#批大小~增大批大小使训练更稳定
batch_size 256
#随机种子一固定种子确保结果可复现
```

play.py中，随机关闭后：

```
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\play.py"
Episodes:   0%|                                                                                                                    | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:52: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider converting the list to a single numpy
.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                                                    | 10/10 [03:26<00:00, 20.62s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```

最终效果：

```
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\train.py"
Episodes: 100%|                                                                                                                    | 1000/1000 [12:14<00:00,  1.36it/s, total_reward=500]
PS E:\LearnAI\aigamelearning> python -u "e:\LearnAI\aigamelearning\play.py"
Episodes:   0%|                                                                                                                    | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:52: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. P
lease consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Trigge
red internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                                                    | 10/10 [03:26<00:00, 20.62s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```



3.Mountain Car：

```
Episodes:   0%|                                                                                                                    | 0/10 [00:00<?, ?it/s]e
:\LearnAI\aigamelearning\play.py:52: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. P
lease consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Trigge
red internally at C:\actions-runner\_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)
  state = torch.tensor([state], dtype=torch.float).to(device)
Episodes: 100%|                                                                                                                    | 10/10 [01:40<00:00, 10.09s/it]
Success rate: 100.00%
PS E:\LearnAI\aigamelearning>
```

思考题解答：

1.Q-learning和DQN：

使用深度Q网络(DQN)算法，因为传统的表格表示法受限于状态和动作数量，所以Q-learning算法基本只能应用在
状态和动作都是离散的、并且空间都比较小的环境。为了处理连续状态空间问题，DQN用深度神经网络来逼近Q函数，并引入了经验回放、目标网络、epsilon-贪婪策略。

2. 推理时的 policy 函数与 DQN 的 take_action 函数的区别

- **DQN 的 take_action 函数**：训练阶段采用**ε- 贪婪策略**，以 ε 概率随机选择动作（探索），以 1-ε 概率选择 Q 值最大的动作（利用），平衡探索与利用。
- **推理时的 policy 函数**：无需探索，仅需利用已学习的策略，因此通常**固定 ε=0**，直接选择 Q 值最大的动作。例如，在 play.py 中实现的 policy 函数仅需根据当前状态计算 Q 值并取最大值对应的动作，无需随机动作。

3. DQN 的 loss 与监督学习 loss 的差异

- **监督学习（如分类）的 loss**：基于**固定标签**（如样本的真实类别），计算预测值与标签的直接差异（如交叉熵损失），目标是最小化当前样本的预测误差。可能会借助对数损失（取对数），借助梯度下降法进行求解。
- DQN 的 loss：

  基于时序差分（TD）目标公式为

  $$L = \mathbb{E}[(y - Q(s,a))^2]$$

  其中目标值

  $$y = r + \gamma \max_{a'} Q'(s', a')$$

  由目标网络计算，包含未来奖励的折扣值。

  1.**动态标签特性**：标签（目标 Q 值）并非固定，而是依赖于环境反馈和未来状态的估值，体现了强化学习的 “延迟奖励” 特性。

  2.**跨时间依赖**：loss 不仅优化当前状态的 Q 值，还间接优化未来状态的估值，通过贝尔曼方程建立时序关联。

4. 机械臂叠方块任务的环境定义

（1）Observation

- **机械臂状态**：关节角度、末端执行器坐标（(x, y, z)）、夹爪开合状态。
- **方块状态**：方块 A 和 B 的位置坐标、姿态（旋转角度）、是否被抓取。
- **特征工程**：可将观测值归一化（如坐标范围映射到 [-1, 1]），或通过传感器数据（如视觉图像）输入神经网络处理。2

（2）Action

- 离散动作空间：

  0：移动机械臂至目标坐标（需指定目标位置）。

  1：抓取方块（若末端执行器靠近方块且未抓取）。

  2：释放方块（若已抓取方块）。

  3：无操作（保持当前状态）。

连续动作空间：直接输出机械臂关节速度或末端执行器的位移增量，应该需要结合一些控制器传感器实现。

（3）Termination

- **成功终止**：方块 A 的底面与方块 B 的顶面接触，且两者位置稳定（如持续 5 步未移动）。

- **失败终止**：
  - 机械臂或方块超出工作空间边界。
  - 尝试抓取时未检测到方块（如夹爪闭合但没有接触物体）。
  - 步数超过最大限制。

（4）Rewards

- **成功奖励**：方块 A 正确叠放至 B 上，给予 + 100 奖励。
- 过程奖励：
  - 靠近目标位置：根据末端执行器与方块 B 顶面的距离给予奖励。
  - 抓取成功：给予 + 20 奖励；抓取失败（如未接触方块）给予 - 10 奖励。
  - 释放成功（方块 A 放置稳定）：给予 + 30 奖励；释放失败（方块 A 掉落）给予 - 20 奖励。
- **惩罚机制**：每步未完成目标时给予 - 1 惩罚，（根据实际完成情况调整reward值）。

5. 训练经验与参数调优

cartpole最难调，其他两个简单

（1）关键参数

- **学习率（lr）**：过高易导致训练震荡，过低则收敛缓慢（如 lr=1e-5 需更多迭代）。在0.01-0.001之间
- **折扣因子（γ）**：γ 接近 1 时更注重长期奖励，适合需要多步规划的任务（如山车问题）；γ 低 时更关注即时奖励，适合短视任务（如 CartPole ）。但是对于长期的也不要太高，短期的也不能太低，0.900-0.955-0.999之间调整
- **探索率（ε）**：
  - 初始 ε=0.9 时探索充分，但初期奖励波动大；
  - ε 退火策略（如线性衰减至 0.1）可平衡初期探索与后期利用。
- **目标网络更新频率**：50比较好，过大与过小在cartpole里面（与50差距>10）就会有明显的效果差异（区间外几乎难以支撑较久）
- **经验回放池容量**：没有调整较多，但是调整比较大会好用些。

（2）调试技巧

- **单一变量调整**：在一开始并不知道具体需要调整的方向时，采取每次仅调整一个参数其余不变方式，对比调整参数后训练模型与调整前的图像与游戏表现（成功率），然后保留比较好的，当一个超参调整基本到达了局部最优（向上、向下调整较小、较大的数值时表现都不如当前情况）观察收敛速度和最终性能差异，确定最优参数区间，之后认为该超参基本调整完成，然后再对其他参数进行单一变量调整，直到每一个超参成功达到当前算法下最优。这是一个比较笨的方法，但是我个人作为新手采取该方法也是成功把cartpole调整到了一个很难死亡的情况（train之中如果不限定游戏模拟轮数就不会停止，表明基本实现了一直不会倾倒）
- **奖励曲线分析**：若曲线波动大，可增大回放池容量或降低学习率；若长期停滞，需调整 ε 退火策略或增加网络复杂度。
- **可视化中间状态**：通过 gym 的 render 模式观察智能体行为，判断是否因探索不足或策略错误导致失败）。

（3）不同游戏环境分析

- **Acrobot**：机械臂主要是一开始启动非常缓慢，参数调整不好时模型几乎不能找到向上摆动的时机导致时间到仍然不能达到较高摆动水平；机器学习到的成功实现策略至少有两个，其一是让主杆晃动幅度逐渐增大，达到转360°，然后小臂被带动晃到线上面；其二是

- **Cartpole**：平衡车主要是很难让模型成功坚持，导致一开始根本学不到什么东西，然而在几次调整未果后参考已有模型参数，然后结合上述不同调整策略，找到一个能坚持一段时间的模型，然后小规模调整达到局部最优。但是仍然难以达到100%正确率，尝试许久后发现参数调整已经达到最好，但是DQN里面优化计算目标Q值(TD误差目标)时算法有一点小问题，调整后能实现稳定不死。并且我们尝试对模型进行“困难模式”，在policy里面故意调整epsilon值很大（即游戏环境很高程度上是随机移动，并非模型自己调整），从而让模型对随机情况进行调整，在其值一定范围内模型还是能撑住的，但是当其过大的时候是不太可能坚持很久（因为train里面是小epsilon，如果训练本身模型的epsilon值和实际比较大时可能会有更好结果？）
- **MountainCar**：在调整玩Cartpole后用该模型直接跑也还可以，然后稍微调整后比较容易上坡。可能游戏环境没有对左侧碰撞限制，导致加速区间比较容易过关，然后调整比较简单。